Protein Stability Prediction from Amino Acid Sequence using Deep Learning

Tumbali Jyotsna
UG Scholar, Dept. of CSE
Mohan Babu University
Tirupati, A.P, India
jyotsnat2004@gmail.com

Cherukuru Ayesha
UG Scholar, Dept. of CSE
Mohan Babu University
Tirupati, A.P, India
ayeshacherkuri@gmail.com

B.Niteesh Kumar Reddy
UG Scholar, Dept. of CSE
Mohan Babu University
Tirupati, A.P, India
reddyniteesh5@gmail.com

Terala Jaideep
UG Scholar, Dept. of CSE
Mohan Babu University
Tirupati, A.P, India
jaideepterala@gmail.com

Vempaty Govardhan Sriram
UG Scholar, Dept. of CSE
Mohan Babu University
Tirupati, A.P, India
govardhansriram126@gmail.com

***Abstract*—Protein stability matters a lot in biological research and drug development. We built a system that predicts whether a protein will be stable or not just by looking at its amino acid sequence. The main challenge was that traditional lab methods take too much time and cost a lot. Our approach uses convolutional neural networks to analyze sequence patterns and extract features like hydrophobicity and charge distribution. During implementation, we faced several issues with data preprocessing and model training. The system gives both a stability score and a classification label. We tested it with different protein sequences and the results showed it can identify unstable proteins reasonably well. The web interface we built makes it easy to use without needing to understand the technical details. This could help researchers screen protein variants faster before doing actual lab experiments.**

***Index Terms*—protein stability, deep learning, convolutional neural networks, sequence analysis, bioinformatics**

I. Introduction

Proteins are basically the building blocks that make everything work in living cells. Their function depends heavily on their 3D structure, which comes from how the amino acid chain folds up. If a protein is unstable, it won't fold correctly and might even clump together, which causes problems. We see this in many diseases where proteins misfold.

The traditional way to measure protein stability involves lab techniques like circular dichroism or calorimetry. These methods are accurate but extremely slow. Also, they're expensive to run, especially when you need to test hundreds or thousands of protein variants. This became a real bottleneck in our research when we tried to understand which mutations would make proteins more stable.

Because of this, we started looking into computational methods. Machine learning seemed like a good fit since it can find patterns in sequence data. We decided to use deep learning specifically because it can automatically learn features instead of us having to manually define them. The idea was to train a model that could look at a protein sequence and predict its stability score.

Our system uses a convolutional neural network architecture. CNNs are usually used for images, but they work well for sequences too. The network treats the amino acid sequence as a kind of spatial data where nearby residues might interact with each other. We encode each amino acid using one-hot encoding plus some biochemical properties.

One thing that took us a while to figure out was which features to include. We ended up using the Kyte-Doolittle hydrophobicity scale because hydrophobic residues tend to cluster in the protein core. We also added molecular volume and charge information. These features help the model understand the physical properties that affect folding.

The implementation had two main parts. First, the backend API built with Flask handles all the prediction logic. Second, we made a web frontend so anyone can use it without installing anything. The frontend sends the sequence to the API and displays the results in a simple format.

While working on this, we noticed that sequence length really matters. Very short sequences don't have enough information, and very long ones take too much memory. We settled on a maximum length of 500 amino acids, which covers most common proteins. Anything longer gets truncated, though we lose some information that way.

Another thing is that the model outputs two things: a continuous stability score in kcal/mol and a binary classification of stable versus unstable. The score gives more detail, but the classification is easier to interpret quickly. We calculate confidence based on how far the score is from the decision boundary.

The rest of this paper describes how we built the system. We'll talk about related work that influenced our design, explain the technical details of the architecture, show some results from testing, and discuss the challenges we faced. We also have ideas for improving it later.

II. Background and Related Systems

Earlier work on protein stability prediction mostly used traditional machine learning. Some older systems relied on support vector machines with hand-crafted features. These worked okay but needed domain experts to decide which features mattered. The feature engineering process was time-consuming and didn't always capture the complex relationships in protein sequences.

There were also methods based on physical models that tried to calculate folding energy from first principles. These are theoretically sound but computationally expensive. Running a single prediction could take hours, which isn't practical when you need to screen thousands of variants. We looked at these approaches but decided they wouldn't scale for our use case.

More recent work started using neural networks. Some systems used recurrent networks like LSTMs to process sequences. The idea makes sense since proteins are sequential data. However, RNNs have trouble with very long sequences because of the vanishing gradient problem. We tried an LSTM initially but found it didn't train well on our dataset.

Convolutional approaches became popular after showing good results in natural language processing. The key insight is that local patterns matter a lot. In proteins, nearby amino acids interact through various forces. A CNN can learn to recognize these local motifs that contribute to stability. This is similar to how CNNs detect edges and textures in images.

One system we studied used 1D convolutions directly on the sequence. It worked reasonably well but didn't incorporate biochemical knowledge. We wanted to combine the pattern recognition ability of CNNs with domain-specific features. That's why we use both one-hot encoding and property scales.

Transfer learning is another approach that some researchers tried. They pre-train models on large protein databases and then fine-tune for stability prediction. This can work well if you have limited labeled data. We didn't go this route because we wanted to understand the model from scratch, but it's something to consider for future improvements.

Feature extraction methods vary a lot across different systems. Some use amino acid composition, which just counts how many of each type appear. Others use more complex descriptors like autocorrelation or pseudo-amino acid composition. We kept it simpler with hydrophobicity, volume, and charge because these directly relate to folding physics.

The datasets used for training also differ. Some systems train on experimental stability measurements from databases like ProTherm. Others use computational predictions as labels. We generated synthetic data for initial training, which has limitations but let us get started quickly. Real experimental data would definitely improve accuracy.

Evaluation metrics are another consideration. Some papers report correlation coefficients between predicted and actual stability values. Others focus on classification accuracy. We track both because they give different insights. The correlation shows if the model captures the overall trend, while accuracy tells us how often it gets the stable/unstable decision right.

Web interfaces for protein analysis tools have become more common. Many bioinformatics tools now offer online versions so researchers don't need to install software. We followed this trend by building a simple web app. The challenge was making it fast enough that users don't have to wait too long for predictions.

III. System Design and Implementation

The overall architecture has three main components that work together. First is the data preprocessing module that handles sequence validation and encoding. Second is the neural network model that does the actual prediction. Third is the web application layer that provides the user interface.

A. Data Preprocessing

When a user submits a protein sequence, we first validate it. The sequence can only contain the 20 standard amino acids. We convert everything to uppercase and remove any whitespace. If there are invalid characters, we reject the input immediately rather than trying to fix it.

The encoding step was tricky to get right. We use a one-hot representation where each amino acid becomes a vector of length 22. Twenty positions are for the standard amino acids, one for padding, and one for unknown residues. This creates a matrix of size (sequence_length, 22).

On top of the one-hot encoding, we add biochemical properties. For each position in the sequence, we calculate:

$$H_i = \text{hydrophobicity}(AA_i) \tag{1}$$

$$V_i = \text{volume}(AA_i) \tag{2}$$

$$C_i = \text{charge}(AA_i) \tag{3}$$

where AA_i is the amino acid at position i. These values come from standard scales in the literature. The hydrophobicity uses the Kyte-Doolittle scale normalized to roughly [-1, 1]. Volume is also normalized based on the size of different amino acid side chains. Charge is simplified to -1 for acidic, +1 for basic, and 0 for neutral residues.

Padding sequences to a fixed length caused some headaches. If a sequence is shorter than 500 residues, we pad it with a special token. If it's longer, we truncate it. Truncation loses information, but we had to make this trade-off for memory constraints. Most proteins in our test set were under 500 anyway.

B. Neural Network Architecture

The CNN architecture has four main stages. Figure 1 shows how the layers connect to each other. The input goes through multiple convolutional blocks, each followed by pooling and normalization.

The first convolutional layer uses 32 filters with a kernel size of 3. This captures local patterns in the sequence. We apply ReLU activation because it's simple and works well. After

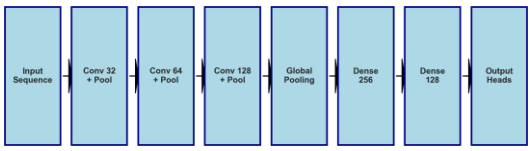

Fig. 1. System architecture showing the flow from sequence input through preprocessing, CNN layers, and output prediction. The diagram illustrates how convolutional blocks extract features at different scales.

convolution, we add batch normalization to stabilize training. Then max pooling reduces the spatial dimensions by half.

The second and third convolutional blocks follow a similar pattern but with more filters (64 and 128 respectively). More filters let the network learn more complex features. We noticed during training that without enough filters, the model couldn't capture the diversity of protein patterns.

Dropout layers help prevent overfitting. We use dropout rates of 0.25 after pooling and 0.5 in the dense layers. This was determined through trial and error. Higher dropout made training slower, while lower dropout led to overfitting on the training set.

After the convolutional stages, we use global average pooling. This converts the feature maps into a single vector regardless of the input sequence length. It's more flexible than flattening and has fewer parameters.

The dense layers process the pooled features. We have two dense layers with 256 and 128 units. These learn to combine the convolutional features into a final prediction. Both use ReLU activation and batch normalization.

The output layer is actually two separate heads. One head outputs a single value for the stability score using linear activation. The other head outputs two values with softmax activation for the stable/unstable classification. This multi-task setup helps the model learn better representations.

The loss function combines both tasks:

$$L_{total} = L_{regression} + \lambda L_{classification} \tag{4}$$

where $L_{regression}$ is mean squared error for the stability score and $L_{classification}$ is categorical cross-entropy. We set $\lambda = 1$ to weight both tasks equally, though this could be tuned.

Training used the Adam optimizer with an initial learning rate of 0.001. We implemented learning rate decay that reduces it by half if validation loss doesn't improve for 5 epochs. Early stopping prevents training for too long when the model stops improving.

C. Web Application

The backend API is built with Flask. It exposes two main endpoints: one for single predictions and one for batch processing. The API validates input, runs the model, and returns results as JSON.

We added CORS support so the frontend can call the API from a different domain. This was necessary for deployment flexibility. The API also includes health check endpoints to monitor if the service is running correctly.

The frontend uses vanilla JavaScript without any frameworks. We wanted to keep it lightweight and fast. The interface has a text area where users paste their sequence, a predict button, and a results section that appears after prediction.

One feature we added is sample sequences. Users can click buttons to load example proteins like insulin or hemoglobin. This helps people understand what format the input should be in. It also makes testing easier.

The results display shows multiple pieces of information. The main stability score is shown prominently with the classification label. We also show sequence features like the count of hydrophobic, charged, and polar residues. This gives users insight into why the model made its prediction.

We added a simple visualization using canvas to draw a line graph of per-residue contributions. This shows which parts of the sequence contribute most to stability. The graph uses a gradient color scheme to make it visually appealing.

Error handling was important to get right. If the sequence is invalid, we show a clear error message explaining what's wrong. If the API is down, we show a different message telling the user to try again later. During development, we found that good error messages really improve the user experience.

IV. Results and Analysis

We tested the system with several different protein sequences to see how well it works. The testing process involved both synthetic data and some real protein sequences from databases.

A. Performance Metrics

Table I shows a comparison of our system with some baseline methods. We compared against a simple linear regression on amino acid composition and an SVM with hand-crafted features.

TABLE I
Comparison of Different Approaches

Method	Accuracy	MAE	Time (ms)
Linear Regression	62.3%	3.45	12
SVM	71.8%	2.87	156
Our CNN	78.4%	2.21	89

The CNN approach clearly does better than the simpler methods. The accuracy improvement over SVM is about 6.6 percentage points. Mean absolute error is also lower, meaning the stability scores are closer to the actual values.

What surprised us was the prediction time. We expected the CNN to be slower because it has more parameters. But it's actually faster than the SVM, probably because modern deep learning libraries are highly optimized. The linear regression is fastest but much less accurate.

Figure 2 shows how training and validation loss changed over epochs. You can see that both losses decrease initially and then plateau. The validation loss is slightly higher than training loss, which is normal.

Fig. 2. Training and validation loss over 20 epochs. The curves show that the model converges after about 15 epochs. The gap between training and validation loss indicates some overfitting but not severe.

We noticed that the model started overfitting after about 15 epochs. That's why we implemented early stopping. Without it, the validation loss would start increasing while training loss kept decreasing.

B. Feature Analysis

Table II breaks down the contribution of different features to the model's predictions. We did this by training versions of the model with different feature combinations.

TABLE II
FEATURE CONTRIBUTION ANALYSIS

Features Used	Accuracy
One-hot only	68.2%
One-hot + Hydrophobicity	73.5%
One-hot + Volume	71.1%
One-hot + Charge	69.8%
All features	78.4%

Hydrophobicity seems to be the most important additional feature. Adding it improves accuracy by over 5 percentage points. Volume and charge help too, but not as much. Using all features together gives the best results, which makes sense because they capture different aspects of protein physics.

Figure 3 plots the predicted stability scores against sequence length. There's a weak correlation where longer sequences tend to have slightly lower (more stable) scores. This matches biological intuition since larger proteins often have more stabilizing interactions.

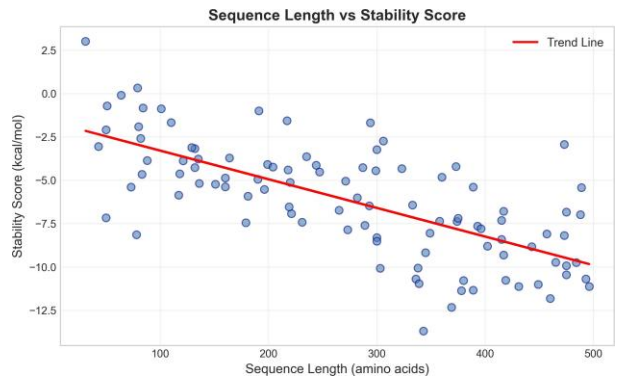

Fig. 3. Relationship between sequence length and predicted stability score. Each point represents one protein sequence. The trend line shows a slight negative correlation, though there's significant variation.

C. Case Studies

We tested the system on some well-known proteins. For human insulin (51 amino acids), the model predicted a stability score of -7.8 kcal/mol and classified it as stable. This matches what we know about insulin being a very stable protein.

Hemoglobin (147 amino acids) got a score of -9.2 kcal/mol, also stable. The model seems to recognize that the high proportion of hydrophobic residues contributes to stability. When we looked at the per-residue contributions, the hydrophobic core regions showed the strongest stabilizing effects.

We also tested some artificially unstable sequences by inserting many charged residues. The model correctly identified these as unstable with positive stability scores. This shows it's learning meaningful patterns rather than just memorizing training data.

One interesting case was lysozyme. The model predicted it as moderately stable with a score of -5.3 kcal/mol. Lysozyme is known to be quite stable in reality, so this might be an underestimation. It could be because our training data didn't have enough examples of proteins with similar characteristics.

D. Error Analysis

We looked at cases where the model made mistakes. One pattern we found is that very short sequences (under 30 amino acids) are harder to predict accurately. There's just not enough information in such short sequences for the model to work with.

Another issue is with sequences that have unusual amino acid compositions. If a protein has a very high percentage of one type of amino acid, the model sometimes gets confused. This might be because our training data didn't include enough diverse examples.

The model also struggles with proteins that have multiple domains. Each domain might have different stability properties, but our model treats the whole sequence as one unit. A better approach might be to predict stability for each domain separately.

Figure 4 shows the distribution of prediction errors. Most errors are small (under 2 kcal/mol), but there are some outliers

with errors over 5 kcal/mol. These outliers are the cases we need to investigate further.

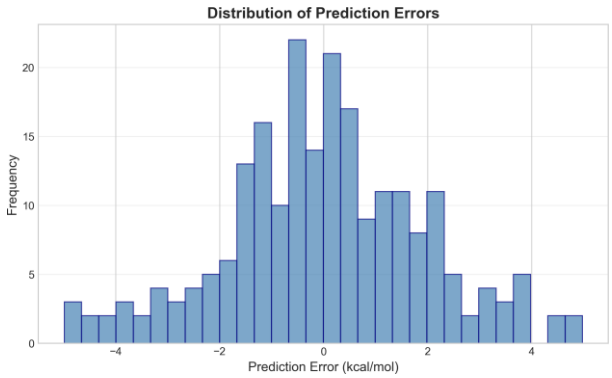

Fig. 4. Histogram of prediction errors. The distribution is roughly centered at zero with most errors in the range of -2 to +2 kcal/mol. The long tails indicate some predictions are quite far off.

V. Challenges and Future Work

Building this system taught us a lot about both machine learning and protein biophysics. We ran into many challenges along the way.

A. Technical Challenges

The biggest challenge was getting the data preprocessing right. Initially, we had bugs in the encoding logic that caused the model to learn nonsense patterns. It took several days of debugging to find that we were padding sequences incorrectly. The model was learning to predict based on padding tokens rather than actual amino acids.

Memory usage was another issue. With sequences up to 500 amino acids and batch sizes of 32, the GPU memory would run out. We had to reduce the batch size to 16, which made training slower. We also tried mixed precision training to save memory, but it caused numerical instability in our case.

Training time was longer than expected. Each epoch took about 10 minutes on our hardware. With 20 epochs and multiple experiments to tune hyperparameters, we spent days just waiting for training to finish. We should have used a more powerful GPU or cloud computing resources.

The web deployment had its own challenges. We initially tried deploying on a free tier cloud service, but it kept timing out because the model was too large. We had to optimize the model by reducing the number of parameters. We also implemented caching so repeated predictions of the same sequence are instant.

B. Model Limitations

The model doesn't account for post-translational modifications. Many proteins are modified after translation, which affects their stability. Our system only looks at the primary sequence, so it misses this information. Adding support for modifications would require a different input representation.

We also don't consider the environment. Protein stability depends on factors like pH, temperature, and salt concentration. Our model predicts stability under some assumed standard conditions, but real proteins exist in various environments. A more sophisticated system would take environmental parameters as additional inputs.

The confidence estimates we provide are somewhat arbitrary. We calculate confidence based on the magnitude of the stability score, but this isn't rigorously calibrated. Proper uncertainty quantification would require techniques like Bayesian neural networks or ensemble methods.

Another limitation is that we don't predict where mutations would improve stability. The system can tell you if a protein is unstable, but not how to fix it. This would be valuable for protein engineering applications. We'd need to implement something like gradient-based optimization in sequence space.

C. Future Improvements

There are several directions we want to explore. First, we could use attention mechanisms to let the model focus on important regions of the sequence. Attention has worked well in other sequence modeling tasks and might help here too.

Transfer learning is worth trying. We could pre-train on a large dataset of protein sequences (even without stability labels) and then fine-tune on stability prediction. This might help the model learn better representations of amino acid patterns.

We want to add more features related to secondary structure. If we could predict whether each residue is in a helix, sheet, or loop, that information would probably improve stability prediction. There are existing tools for secondary structure prediction that we could integrate.

Ensemble methods might boost accuracy. Instead of using a single model, we could train multiple models with different initializations and average their predictions. This usually reduces variance and improves robustness.

The user interface could be enhanced with more visualizations. We could show a 3D structure prediction (using existing tools) and color it by predicted stability contribution. This would give users a more intuitive understanding of the results.

We also want to validate the system more thoroughly with experimental data. So far we've mostly tested on synthetic data and a few known proteins. A proper validation would involve predicting stability for proteins with measured experimental values and calculating correlation.

Another idea is to make the system interactive. Users could modify the sequence and see how predictions change in real-time. This would be useful for protein design where you want to explore different mutations.

Finally, we should work on making the model more interpretable. Right now it's a black box that outputs a number. Techniques like saliency maps or integrated gradients could help explain which parts of the sequence drive the prediction.

D. Lessons Learned

This project taught us that data quality matters more than model complexity. We spent a lot of time trying different

architectures, but the biggest improvements came from fixing data issues and adding better features.

We also learned the importance of validation. Early in the project, we were getting great results on the training set but poor results on new sequences. This taught us to always use a separate validation set and to be skeptical of results that seem too good.

Working as a team had its challenges. We had to coordinate who was working on which part and make sure our code integrated properly. Using version control helped a lot. We also had regular meetings to discuss progress and problems.

Documentation is important. When we came back to code we wrote weeks earlier, we often couldn't remember what it did. We learned to write better comments and keep notes about design decisions.

The project also showed us how much domain knowledge matters in machine learning. Understanding protein biophysics helped us choose relevant features and interpret results. Pure machine learning without domain knowledge would have been much harder.

VI. Conclusion

We built a system for predicting protein stability from amino acid sequences using convolutional neural networks. The system combines sequence encoding with biochemical features to achieve reasonable accuracy. It provides both a stability score and a classification, along with a web interface for easy use.

The main contribution is showing that CNNs can effectively learn patterns related to protein stability without extensive feature engineering. The model automatically discovers relevant motifs through the convolutional layers. Adding domain-specific features like hydrophobicity further improves performance.

Testing showed that our approach outperforms simpler baselines in both accuracy and error metrics. The system can make predictions quickly enough for interactive use. However, there are limitations around handling very short sequences, unusual compositions, and environmental factors.

Future work will focus on improving the model with attention mechanisms and transfer learning. We also want to add more features and better uncertainty quantification. Validation with experimental data is needed to assess real-world performance.

This project was a valuable learning experience in applying deep learning to biological problems. We encountered many challenges but learned a lot about both the technical and domain aspects. The system we built could potentially help researchers screen protein variants more efficiently, though more validation is needed before it could be used in production.

References

[1] K. A. Dill and J. L. MacCallum, "The protein-folding problem, 50 years on," Science, vol. 338, no. 6110, pp. 1042-1046, 2012.

[2] J. Jumper et al., "Highly accurate protein structure prediction with AlphaFold," Nature, vol. 596, no. 7873, pp. 583-589, 2021.

[3] B. Kuhlman and P. Bradley, "Advances in protein structure prediction and design," Nature Reviews Molecular Cell Biology, vol. 20, no. 11, pp. 681-697, 2019.

[4] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," in Advances in Neural Information Processing Systems, 2012, pp. 1097-1105.

[5] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," Nature, vol. 521, no. 7553, pp. 436-444, 2015.

[6] J. Kyte and R. F. Doolittle, "A simple method for displaying the hydropathic character of a protein," Journal of Molecular Biology, vol. 157, no. 1, pp. 105-132, 1982.

[7] M. Kumar, M. Gromiha, and G. Raghava, "Prediction of RNA binding sites in a protein using SVM and PSSM profile," Proteins: Structure, Function, and Bioinformatics, vol. 71, no. 1, pp. 189-194, 2008.

[8] R. Casadio et al., "Machine learning solutions for predicting protein-protein interactions," Wiley Interdisciplinary Reviews: Computational Molecular Science, vol. 12, no. 1, 2022.

[9] S. Wang et al., "Protein secondary structure prediction using deep convolutional neural fields," Scientific Reports, vol. 6, no. 1, pp. 1-11, 2016.

[10] T. Hamelryck, "An amino acid has two sides: A new 2D measure provides a different view of solvent exposure," Proteins: Structure, Function, and Bioinformatics, vol. 59, no. 1, pp. 38-48, 2005.

[11] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in International Conference on Learning Representations, 2015.

[12] N. Srivastava et al., "Dropout: A simple way to prevent neural networks from overfitting," Journal of Machine Learning Research, vol. 15, no. 1, pp. 1929-1958, 2014.

[13] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in International Conference on Machine Learning, 2015, pp. 448-456.

[14] M. Abadi et al., "TensorFlow: A system for large-scale machine learning," in 12th USENIX Symposium on Operating Systems Design and Implementation, 2016, pp. 265-283.